QSM603FCP Ubuntu 外设驱动开发指导

智能产品

版本：1.0.0

日期：2026-10-14

状态：临时文件

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市松江区泗泾镇外婆泾路 8 号　邮编：201601
电话：+86 21 5108 6236　　邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：https://www.quectel.com.cn/contact。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
https://www.quectel.com.cn/contact?tab=t 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持您的产品设计。您须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。您知悉并同意，移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档，增加、修改或重述后的文档对您具有约束力。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，任何形式的购买都不可被视为授予除正常的非独家、免版税的产品使用许可之外的任何许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法、侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何明示、暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2026，保留一切权利。
Copyright © Quectel Wireless Solutions Co., Ltd. 2026.

文档历史

修订记录

版本	日期	作者	变更描述
-	2026-10-14	Levi Cao/ Dean Zhou/ Faber Zhong	文档创建
1.0.0	2026-10-14	Levi Cao/ Dean Zhou/ Faber Zhong	临时版本

目录

文档历史 3
目录 4
表格索引 6
图片索引 7

1 引言 8

2 ADC 功能 9
2.1. ADC 简介 9
2.2. ADC 使用 9
2.2.1. ADC 代码文件路径 9
2.2.2. API 函数介绍 10
2.2.3. 在内核中使用 ADC 11
2.3. ADC 节点使用示例 12

3 SPI 功能 13
3.1. SPI 接口引脚定义 13
3.2. Kernel 阶段 SPI 相关配置 14
3.2.1. SPI 控制器节点配置 14
3.2.2. 添加 SPI Slave 设备参考配置 15
3.3. 内核中常用 SPI 接口函数 16

4 UART 功能 17
4.1. 内核中配置 UART 18
4.1.1. 配置 UART 内核设备节点 18
4.1.2. 配置 UART Pinctrl 18
4.1.3. 启用 UART 配置 19
4.1.4. 验证 UART 用户接口节点 19
4.2. 用户空间中配置 UART 示例 20

5 USB 功能 23
5.1. USB 架构图 23
5.2. USB 使用场景 24
5.3. USB 软件驱动程序 24
5.4. USB 调试方法 25
5.4.1. 查看 USB 当前工作模式 25
5.4.2. 主机模式下查看 USB 外设连接信息 25

6 充电功能 26

7 I2C 功能 27
7.1. 设备 I2C 配置信息 27
7.2. 配置文件介绍 28
7.3. 内核中 I2C 客户端配置 32
7.3.1. 配置 I2C 客户端设备 32

7.3.2. 验证 I2C 客户端配置结果 33
7.4. 内核 I2C 接口函数 34
7.5. 用户空间中操作 I2C 示例 34
7.5.1. 调用 I2C 相关接口读写数据 34
7.5.2. 调用底层 ioctl 接口读写数据 35

8 GPIO 功能 37
8.1. GPIO 简介 37
8.2. 在内核中操作 GPIO 38
8.2.1. 在设备树中定义 GPIO 编号 38
8.2.2. 在内核驱动中包含 GPIO 头文件 39
8.2.3. 从设备树定义中获取 GPIO 编号 39
8.2.4. 配置 GPIO 为输入模式 39
8.2.5. 配置 GPIO 为输出模式 39
8.2.6. 在内核驱动中设置 GPIO 输出电平 39
8.3. 使用 pinctrl 配置 GPIO 40
8.3.1. 在设备树中定义 GPIO 的 pinctrl 属性 40
8.3.2. 在内核驱动中包含头文件 40
8.3.3. 在内核驱动中获取 pinctrl 及其状态 40
8.3.4. 在内核驱动中选择 pinctrl 状态 41

9 PCIe 功能 42
9.1. 功能概述 42
9.2. PCIe 能力说明 42
9.3. PCIe 控制器节点 43
9.4. PCIe 控制器驱动 44

10 附录 参考文档和术语缩写 45

表格索引

表 1：SPI 接口引脚定义 13
表 2：支持的 UART 配置 17
表 3：USB 端口信息 24
表 4：USB 软件驱动程序 24
表 5：I2C 配置信息 27
表 6：参考文档 45
表 7：术语缩写 45

图片索引

图 1：ADC 节点位置 9
图 2：USB 架构 23
图 3：I2C 节点位置 28
图 4：设备 PCIe 至 YT6801 通路图 42

1 引言

本文档主要介绍如何在移远通信 QSM603FCP 设备（以下简称“设备”）上配置外设相关功能，如 ADC 功能、SPI 功能、UART 功能、USB 功能、充电功能、I2C 功能、GPIO 功能以及 PCIe 功能。

备注

1. 本文档适用于运行 Ubuntu 操作系统的 QSM603FCP 设备。
2. 由于 PDF 格式特性，文档中部分代码块可能因页面宽度限制自动换行。直接从 PDF 复制代码到编辑器时，可能引入非预期的换行符（\n），导致代码无法正常运行。

   建议操作：

   若需从 PDF 复制，粘贴后请注意完成如下动作：
   - 手动删除复制粘贴后代码中的多余换行符。
   - 或通过代码编辑器的替换功能批量删除多余换行符（注意保留语义换行）。

2 ADC 功能

本章节主要描述设备的 ADC 功能，并介绍如何使用设备提供的 ADC 接口。

2.1. ADC 简介

设备共有 16 路 ADC：ADC0 ~ ADC15，其中只引出 ADC1 ~ ADC4 共 4 路 ADC 供用户使用（可同时使用），其他 ADC 为内部专用 ADC，暂时不开放给用户。ADC 采样速率支持 0.2 MSPS，采样精度为 12 位。

设备默认已初始化所有 ADC，各 ADC 节点的路径为*/sys/bus/iio/devices/iio:device1*，如下所示：

```
sh-5.2# pwd
/sys/bus/iio/devices/iio:device1
sh-5.2# ls
in_voltage0_input   in_voltage15_input  in_voltage6_input  power
in_voltage10_input  in_voltage1_input   in_voltage7_input  subsystem
in_voltage11_input  in_voltage2_input   in_voltage8_input  uevent
in_voltage12_input  in_voltage3_input   in_voltage9_input  waiting_for_supplier
in_voltage13_input  in_voltage4_input   name
in_voltage14_input  in_voltage5_input   of_node
sh-5.2#
```

图 1：ADC 节点位置

供外部使用的 ADC1 ~ ADC4 对应上图中的 *in_voltage1~4_**，其中：

- *后缀为 *input* 时表示 ADC 获取的输入电压值；
- *后缀为 *raw* 时表示原始采样值。

2.2. ADC 使用

2.2.1. ADC 代码文件路径

头文件路径：
sources/linux-mtk/include/linux/iio/consumer.h

源码文件路径：
sources/linux-mtk/drivers/iio/adc/mt6577_auxadc.c

DTSI 文件路径：
sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8189.dtsi

2.2.2. API 函数介绍

2.2.2.1. iio_channel_get

该函数用于获取访问通道所需的信息。

- **函数原型**

```
iio_channel_get (struct device * dev, const char * consumer_channel)
```

- **参数**

dev：
[In] 用户设备指针。

consumer_channel：
[Out] 用户端通道的 ID。

- **返回值**

函数执行成功，则返回与 ADC 通道地址对应的 IIO 通道结构；函数执行失败则返回负值。

2.2.2.2. iio_read_channel_processed

该函数用于获取对应 ADC 通道的电压值。

- **函数原型**

```
io_read_channel_processed (struct iio_channel * chan, int * val)
```

- **参数**

chan：
[In] 查询的通道。

val：
[Out] 实际返回的电压值。

- **返回值**

0　　函数执行成功
其他值　函数执行失败

2.2.2.3. iio_channel_release

该函数用于释放通过 *iio_channel_get()*获取到的通道。

- **函数原型**

```
iio_channel_release (struct iio_channel * chan)
```

- **参数**

chan：
[In] 要被释放的通道。

- **返回值**

无

2.2.3. 在内核中使用 ADC

1. 定义一个 iio_channel 结构体参数。

```
#include <linux/iio/consumer.h>
struct iio_channel *thermistor_ch0;
```

2. 在 DTS 文件中通过 *io-channels* 配置使用的 ADC 通道，通过 *io-channel-names* 配置该 ADC 通道的名称，如下所示：

```
tboard_thermistor1: thermal-sensor1 {
        compatible = "mediatek,mtboard-thermistor1";
        io-channels = <&auxadc 3>;
        io-channel-names = "thermistor-ch3";
        interconnects = <&auxadc 3>;
    };
```

- *io-channels = <&auxadc 3>*表示配置使用的 ADC 通道为 ADC3；
- *io-channel-names = "thermistor-ch3"*表示配置该 ADC 通道的名称为“thermistor-ch3”。

3. 通过 Probe 函数根据 DTS 配置获取 iio_channel 句柄。

```
thermistor_ch3 = devm_kzalloc(&pdev->dev, sizeof(*thermistor_ch3),GFP_KERNEL);
thermistor_ch3 = iio_channel_get(&pdev->dev, "thermistor-ch3");
```

4. 获取该 ADC 通道电压值。

```
ret = iio_read_channel_processed(thermistor_ch3, &val);
```

备注

电池温度检测所使用的 ADC 通道必须正确连接并配置，否则系统可能无法正常开机。

2.3. ADC 节点使用示例

ADC IIO 设备节点用于读取对应通道的输入电压值。用户可以通过 **cat** 命令读取节点获取 ADC 采样结果，例如：

```
cat sys/bus/iio/devices/iio:device1/in_voltage3_input
```

说明：

- *iio:device1* 表示 IIO 设备编号（实际编号可能不同）；
- *in_voltage3_input* 表示读取第 3 路 ADC 通道的输入电压。

注意返回的数值单位为 mV（毫伏）。

3 SPI 功能

本章节主要介绍如何在设备上配置 SPI，包括如何在内核中配置 SPI 和常用 SPI 接口函数介绍。

3.1. SPI 接口引脚定义

设备共支持 5 组 SPI（可同时使用），支持最大传输速率为 52 MHz。其中 3 组为通用 SPI 接口，详情见下表，用户可根据需求配置；其余 2 组 SPI 由其他接口复用。

表 1：SPI 接口引脚定义

接口号	GPIO 号	引脚号	I/O	引脚复用	引脚描述
SPI0	GPIO_69	214	DO	SPI0_CSB	SPI0 片选
	GPIO_71	217	DO	SPI0_MO	SPI0 数据输出
	GPIO_72	216	DI	SPI0_MI	SPI0 数据输入
	GPIO_70	215	DO	SPI0_CLK	SPI0 时钟
SP1	GPIO_73	208	DO	SPI1_CSB	SPI1 片选
	GPIO_75	210	DO	SPI1_MO	SPI1 数据输出
	GPIO_76	211	DI	SPI1_MI	SPI1 数据输入
	GPIO_74	209	DO	SPI1_CLK	SPI1 时钟
SPI2	GPIO_77	262	DO	SPI2_CSB	SPI2 片选
	GPIO_79	263	DO	SPI2_MO	SPI2 数据输出
	GPIO_80	264	DI	SPI2_MI	SPI2 数据输入
	GPIO_78	261	DO	SPI2_CLK	SPI2 时钟

备注

若相关引脚已被配置为其他功能，软件要配置 SPI，需要确认硬件上对应的引脚没有用作其它功能。

3.2. Kernel 阶段 SPI 相关配置

本章主要介绍如何在 kernel 阶段配置 SPI。

3.2.1. SPI 控制器节点配置

SPI 的控制器节点位于：

- *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8189.dtsi*
- *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts*

3.2.1.1. 配置 SPI0 控制器节点

```
spi0: spi@11010800 {
        compatible = "mediatek,mt8189-spi";
        mediatek,pad-select = <0>;
        reg = <0 0x11010800 0 0x100>;
        interrupts = <GIC_SPI 459 IRQ_TYPE_LEVEL_HIGH 0>;
        clocks = <&topckgen_clk CLK_TOP_UNIVPLL_D6_D2>,
                 <&topckgen_clk CLK_TOP_SPI0_SEL>,
                 <&pericfg_ao_clk CLK_PERAO_SPI0_B>,
                 <&pericfg_ao_clk CLK_PERAO_SPI0_H>;
        clock-names = "parent-clk", "sel-clk", "spi-clk", "hclk";
        #address-cells = <1>;
        #size-cells = <0>;
};
```

3.2.1.2. 配置 SPI1 控制器节点

```
spi1: spi@11011800 {
        compatible = "mediatek,mt8189-spi";
        mediatek,pad-select = <0>;
        reg = <0 0x11011800 0 0x100>;
        interrupts = <GIC_SPI 460 IRQ_TYPE_LEVEL_HIGH 0>;
```

```
        clocks = <&topckgen_clk CLK_TOP_UNIVPLL_D6_D2>,
                 <&topckgen_clk CLK_TOP_SPI1_SEL>,
                 <&pericfg_ao_clk CLK_PERAO_SPI1_B>,
                 <&pericfg_ao_clk CLK_PERAO_SPI1_H>;
        clock-names = "parent-clk", "sel-clk", "spi-clk", "hclk";
        #address-cells = <1>;
        #size-cells = <0>;
};
```

3.2.1.3. 配置 SPI2 控制器节点

```
spi2: spi@11012800 {
        compatible = "mediatek,mt8189-spi";
        mediatek,pad-select = <0>;
        reg = <0 0x11012800 0 0x100>;
        interrupts = <GIC_SPI 461 IRQ_TYPE_LEVEL_HIGH 0>;
        clocks = <&topckgen_clk CLK_TOP_UNIVPLL_D6_D2>,
                 <&topckgen_clk CLK_TOP_SPI2_SEL>,
                 <&pericfg_ao_clk CLK_PERAO_SPI2_B>,
                 <&pericfg_ao_clk CLK_PERAO_SPI2_H>;
        clock-names = "parent-clk", "sel-clk", "spi-clk", "hclk";
        #address-cells = <1>;
        #size-cells = <0>;
};
```

3.2.2. 添加 SPI Slave 设备参考配置

本节以 SPI0 为例说明 Slave 设备的添加流程。

1. 在板级 DTS 文件 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 中为 SPI 主控添加从设备节点（示例）：

```
&spi0 {
        pinctrl-0 = <&spi0_pins>;
        pinctrl-names = "default";
        #address-cells = <1>;
        #size-cells = <0>;
        status = "okay";

        spi_dev0@0 {
                compatible = "vendor,spi-dev-demo";
                reg = <0>;
```

```
            spi-max-frequency = <12000000>;
        };
};
```

用户必须配置 *compatible*、*reg* 和 *spi-max-frequency* 参数，参数值由用户自定义。对应参数描述如下：

- *compatible*：用于匹配驱动代码。
- *reg*：片选设备，例如，示例中<0>表示片选从设备 0。
- *spi-max-frequency*：SPI 设备的最大频率，必须小于或者等于 SPI 主机的 *spi-max-frequency* 属性值。

备注

当前板级设备树文件中，*&spi0* 节点下已挂载了一个 CAN 从设备节点 *can0@0*。

2. 参考***文档 [1]***进行编译和烧录。编译烧录后，会在*/sys/devices/platform/soc/11010800.spi0/spi_master/spi0* 路径下生成 *spi0.0* 节点，如下图所示：



3.3. 内核中常用 SPI 接口函数

内核中常用的标准 SPI 接口函数如下，具体使用案例请参考源码。

```
//开始建立 SPI 通信。
int spi_setup(struct spi_device *spi);
//通过 SPI 写入数据。
static inline int spi_write(struct spi_device *spi, const void *buf, size_t len);
//通过 SPI 读取数据。
static inline int spi_read(struct spi_device *spi, void *buf, size_t len);
//SPI 数据同步传输。
static inline int spi_sync_transfer(struct spi_device *spi, struct spi_transfer *xfers, unsigned int num_xfers);
//通过 SPI 先写入数据，然后再读取数据。
int spi_write_then_read(struct spi_device *spi, const void *txbuf, unsigned n_tx, void *rxbuf, unsigned n_rx);
```

4 UART 功能

本章节主要介绍在设备上配置 UART 的方法，以及在内核及用户空间中操作 UART 的示例。

设备共支持 4 组 UART（包含 1 组 Debug UART），可同时使用，支持最大传输速率 921600 bps。用户可按需选配 UART。UART 配置信息如下：

表 2：支持的 UART 配置

UART 名称	文件名	物理地址	引脚名称	PIN 号	GPIO 号
Debug UART	*/dev/tty**	0x11001000	DBG_TXD	46	GPIO31
			DBG_RXD	45	GPIO32
UART1	*/dev/tty**	0x11002000	UART1_TXD	13	GPIO33
			UART1_RXD	14	GPIO34
UART2	*/dev/tty**	0x11003000	UART2_TXD	229	GPIO35
			UART2_RXD	230	GPIO36
			UART2_RTS	227	GPIO17
			UART2_CTS	228	GPIO16
UART3	*/dev/tty**	0x11004000	UART3_TXD	15	GPIO37
			UART3_RXD	16	GPIO38
			UART3_RTS	160	GPIO15
			UART3_CTS	161	GPIO14

4.1. 内核中配置 UART

下面以 UART2 为例，介绍 UART 的相关配置流程，其他 UART 的配置流程可参考该配置。

4.1.1. 配置 UART 内核设备节点

在 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8189.dtsi* 文件中，添加如下 UART 设备节点：

```
uart2: serial@11003000 {
	compatible = "mediatek,mt6577-uart";
	reg = <0 0x11003000 0 0x1000>;
	interrupts = <GIC_SPI 399 IRQ_TYPE_LEVEL_HIGH 0>;
	clocks = <&clk26m>, <&pericfg_ao_clk CLK_PERAO_UART2>;
	clock-names = "baud", "bus";
	dmas = <&apdma 4 &apdma 5>;
	dma-names = "tx", "rx";
	uart-line = <2>;
	status = "disabled";
};
```

4.1.2. 配置 UART Pinctrl

在 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 文件中，添加如下 UART Pinctrl 配置：

```
&pio{
	uart2_pins: uart2-pins {
		pins {
			pinmux = <PINMUX_GPIO35__FUNC_UTXD2>,
				 <PINMUX_GPIO36__FUNC_URXD2>;
			bias-pull-up;
		};

		rtscts-pins {
			pinmux = <PINMUX_GPIO16__FUNC_UCTS2>,
				 <PINMUX_GPIO17__FUNC_URTS2>;
		};
	};
}
```

4.1.3. 启用 UART 配置

在 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 文件中，添加如下配置以启用 UART2：

```
&uart2 {
	pinctrl-0 = <&uart2_pins>;
	pinctrl-names = "default";
	status = "okay";
};
```

4.1.4. 验证 UART 用户接口节点

当设备配置了多个 UART 时，用户空间下将存在多个 UART 接口节点，如下图所示：

```
sh-5.2# pwd
/dev
sh-5.2# ls tty*
tty    tty15  tty22  tty3   tty37  tty44  tty51  tty59  tty9    ttyp2  ttypa
tty0   tty16  tty23  tty30  tty38  tty45  tty52  tty6   ttyGS0  ttyp3  ttypb
tty1   tty17  tty24  tty31  tty39  tty46  tty53  tty60  ttyS0   ttyp4  ttypc
tty10  tty18  tty25  tty32  tty4   tty47  tty54  tty61  ttyS1   ttyp5  ttypd
tty11  tty19  tty26  tty33  tty40  tty48  tty55  tty62  ttyS2   ttyp6  ttype
tty12  tty2   tty27  tty34  tty41  tty49  tty56  tty63  ttyS3   ttyp7  ttypf
tty13  tty20  tty28  tty35  tty42  tty5   tty57  tty7   ttyp0   ttyp8
tty14  tty21  tty29  tty36  tty43  tty50  tty58  tty8   ttyp1   ttyp9
```

此时无法直接确定哪个节点对应 UART2 所生成的接口，需要到*/sys/class/tty* 路径下使用 **ls -al** 查看节点地址：

```
sh-5.2# pwd
/sys/class/tty
sh-5.2# ls
console  ptyp8  tty1   tty19  tty28  tty37  tty46  tty55  tty7    ttyp2  ttypc
ptmx     ptyp9  tty10  tty2   tty29  tty38  tty47  tty56  tty8    ttyp3  ttypd
ptyp0    ptypa  tty11  tty20  tty3   tty39  tty48  tty57  tty9    ttyp4  ttype
ptyp1    ptypb  tty12  tty21  tty30  tty4   tty49  tty58  ttyGS0  ttyp5  ttypf
ptyp2    ptypc  tty13  tty22  tty31  tty40  tty5   tty59  ttyS0   ttyp6
ptyp3    ptypd  tty14  tty23  tty32  tty41  tty50  tty6   ttyS1   ttyp7
ptyp4    ptype  tty15  tty24  tty33  tty42  tty51  tty60  ttyS2   ttyp8
ptyp5    ptypf  tty16  tty25  tty34  tty43  tty52  tty61  ttyS3   ttyp9
ptyp6    tty    tty17  tty26  tty35  tty44  tty53  tty62  ttyp0   ttypa
ptyp7    tty0   tty18  tty27  tty36  tty45  tty54  tty63  ttyp1   ttypb
sh-5.2# ls -al ./tty*/device
lrwxrwxrwx 1 root root 0 May 30 02:54 ./ttyS0/device -> ../../../11001000.serial
lrwxrwxrwx 1 root root 0 May 30 02:54 ./ttyS1/device -> ../../../11002000.serial
lrwxrwxrwx 1 root root 0 May 30 02:51 ./ttyS2/device -> ../../../11003000.serial
lrwxrwxrwx 1 root root 0 May 30 02:54 ./ttyS3/device -> ../../../11004000.serial
```

由输出结果可知，地址为 11003000 的串口节点对应为 *ttyS2*，因此用户空间下的 UART2 串口节点即为*/dev/ttyS2*。

常用的串口节点交互命令如下：

- 查看串口节点的配置：**stty -F /dev/ttyS2**
- 设置串口节点的波特率：**stty -F /dev/ttyS2 [baudrate]**
- 接收串口节点数据：**cat /dev/ttyS2**
- 向串口节点发送数据：**echo [data] > /dev/ttyS2**

4.2. 用户空间中配置 UART 示例

在用户空间中使用 UART 读写数据的示例如下：

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <termios.h>
#include <sys/types.h>
#include <pthread.h>

#define SPEED B115200
#define BUF_SIZE (4 * 1024)

static int open_port(char *node)
{
  int fd;
  struct termios options;

  fd = open(node, O_RDWR | O_NOCTTY | O_NDELAY);
  if (fd < 0) {
printf("Could not open %s\n", node);
return -1;
  }
  if (tcflush(fd, TCIOFLUSH) < 0) {
    printf("Could not flush uart port");
    return -1;
  }

  if (tcgetattr(fd, &options) < 0) {
```

```
    printf("Can't get port settings \n");
    return -1;
  }

  options.c_cc[VTIME]      = 0;
  options.c_cc[VMIN]       = 1;

  options.c_cflag &= ~CSIZE;
  options.c_cflag |= (CS8 | CLOCAL | CREAD);
  options.c_iflag = IGNPAR;
  options.c_oflag = 0;
  options.c_lflag = 0;
  cfsetospeed(&options, B115200);
  cfsetispeed(&options, B115200);
  if (tcsetattr(fd, TCSANOW, &options) < 0) {
    printf("Can't set port setting \n");
    return -1;
  }

  fcntl(fd, F_SETFL, 0);

  return fd;
}

void clear_buf(unsigned char *buf)
{
  int i;

  for(i = 0; i < BUF_SIZE; i++)
    buf[i] = 0x00;
}

int main(int argc, char *argv[])
{
  int fd;
  unsigned char buf[BUF_SIZE];

  if(argc != 2) {
    printf("usage:%s /dev/ttyHSLx\n", argv[0]);
    return 0;
  }

  fd = open_port(argv[1]);  if(fd < 0)
    return -1;
```

```
    while(1)   {
        read(fd,   buf,   BUF_SIZE);
        write(fd,   buf,   strlen(buf));
        clear_buf(buf);
    }

    close(fd);

    return   0;
}
```

5 USB 功能

本章节主要提供了设备 USB 介绍、实现方式及调试方法。

QSM603FCP 设备提供 5 路 USB 接口（P0~P4），支持同时使用。

其中，P0 和 P3 支持 USB 3.0（向下兼容 USB 2.0）：

- P0 可配置为 Host 模式或 Device 模式；
- P3 仅支持 Host 模式。

P1、P2 和 P4 仅支持 USB 2.0 Host 模式。

在速率方面，P0 和 P3 在 USB 3.0 模式下最高支持 5 Gbps，而 P1、P2 和 P4 在 USB 2.0 模式下最高支持 480 Mbps。

5.1. USB 架构图

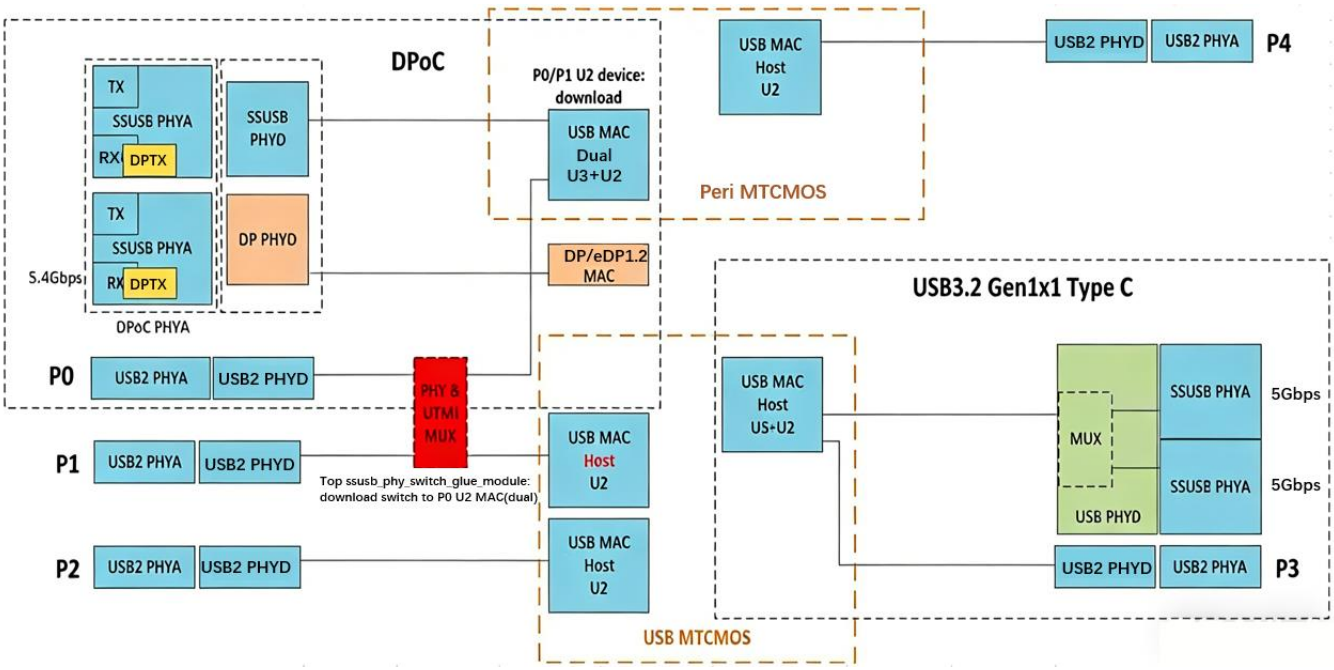


图 2：USB 架构

表 3：USB 端口信息

USB 端口	备注
Port 0	USB 3.2 Gen 1 双角色模式（DRD），与 DP1.4-TX 复用
Port 1	USB 2.0 主模式
Port 2	USB 2.0 主模式
Port 3	USB 3.2 Gen 1 主模式
Port 4	USB 2.0 主模式

5.2. USB 使用场景

1. 紧急下载模式（PBL 阶段）；
2. ADB、diag 和 Vcom；
3. 多媒体传输协议（MTP）、图片传输协议（PTP）和大容量存储；
4. 远程网络接口规范（RNDIS）；
5. 识别 HUB、人体接口设备和大容量存储设备等；
6. USB 充电。

5.3. USB 软件驱动程序

表 4：USB 软件驱动程序

主驱动	位置	描述
mt8189.dtsi	*target/mtk/mt8371/source/linux-mtk-6.8/arch/arm64/boot/dts/mediatek/mt8189.dtsi*	当前设备使用的主设备树
mt8391-genio-720-evk.dts	*target/mtk/mt8371/source/linux-mtk-6.8/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts*	用于包含修改配置的设备树
tcpm.c	*target/mtk/mt8371/source/linux-mtk-6.8/drivers/usb/typec/tcpm/tcpm.c*	Type-C 协议相关的驱动源码

tcpci	*target/mtk/mt8371/source/linux-mtk-6.8/drivers/usb/typec/tcpm/tcpci.c*	Type-C 协议相关的驱动源码

5.4. USB 调试方法

5.4.1. 查看 USB 当前工作模式

执行如下命令：

```
cat /sys/class/usb_role/11201000.usb0-role-switch/role
```

USB 有两种工作模式：Device 模式和 Host 模式。

命令执行结果中：

- *Device* 表示 USB 处于从机模式；
- *Host* 表示 USB 处于主机模式；

5.4.2. 主机模式下查看 USB 外设连接信息

执行以下命令查看挂载的 USB 设备详情：

```
cat /sys/kernel/debug/usb/devices
```

命令执行结果如下：

```
T:  Bus=01 Lev=00 Prnt=00 Port=00 Cnt=00 Dev#=  1 Spd=480  MxCh= 1
B:  Alloc=  0/800 us ( 0%), #Int=  0, #Iso=  0
D:  Ver= 2.00 Cls=09(hub  ) Sub=00 Prot=01 MxPS=64 #Cfgs=  1
P:  Vendor=1d6b ProdID=0002 Rev= 4.19
S:  Manufacturer=Linux 4.19.191 musb-hcd
S:  Product=MUSB HDRC host driver
S:  SerialNumber=musb-hdrc
C:* #Ifs= 1 Cfg#= 1 Atr=e0 MxPwr=  0mA
I:* If#= 0 Alt= 0 #EPs= 1 Cls=09(hub  ) Sub=00 Prot=00 Driver=hub
E:  Ad=81(I) Atr=03(Int.) MxPS=  4 Ivl=256ms
```

如上所示，Spd=480 表示 USB 2.0 外设。若 Spd=5000，则表示 USB 3.0 外设。

6 充电功能

QSM603FCP 设备不支持 USB 口充电，采用 DC 供电，并配有 UPS 电池。硬件侧已实现充电控制逻辑，软件侧只需按硬件定义使能对应的充电控制 GPIO，无需其他修改。

固定 GPIO 电源配置路径：

target/mtk/mt8371/source/linux-mtk-6.8/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts

```
/* GPIO109 is the active-low UPS hard-charge enable on this board. */
	regulator-ups-hard-charge {
		compatible = "regulator-fixed";
		regulator-name = "ups-hard-charge";
		regulator-min-microvolt = <5000000>;
		regulator-max-microvolt = <5000000>;
		gpio = <&pio 109 GPIO_ACTIVE_LOW>;
		regulator-boot-on;
		regulator-always-on;
	};
```

GPIO 使能操作路径：

target/mtk/mt8371/source/u-boot/arch/arm/dts/genio-720-evk-common.dtsi

```
/* GPIO109 drives the active-low UPS hard-charge enable. */
	ups-hard-charge-enable-hog {
		gpio-hog;
		gpios = <109 GPIO_ACTIVE_HIGH>;
		output-low;
		line-name = "ups-hard-charge-enable";
	};
```

7 I2C 功能

本章节介绍如何在设备上配置 I2C，包括如何在内核中配置 I2C、常用 I2C 接口函数介绍，以及用户空间中操作 I2C 的方法。

7.1. 设备 I2C 配置信息

QSM603FCP 设备有 8 路 I2C，支持 8 路 I2C 同时使用。支持标准模式（100 KB/s）、快速模式（400 KB/s）、快速增强模式（1 MB/s）和高速模式（3.4 MB/s）。I2C 配置信息如下表所示：

表 5：I2C 配置信息

序号	引脚号	引脚名称	引脚复用
1	366	SENSOR_I2C0_SCL	SCP_SCL0
	367	SENSOR_I2C0_SDA	SCP_SDA0
2	220	SENSOR_I2C1_SCL	SCP_SCL1
	219	SENSOR_I2C1_SDA	SCP_SDA1
3	309	I2C2_SCL	SCL2
	310	I2C2_SDA	SDA2
4	272	I2C3_SCL	SCL3
	271	I2C3_SDA	SDA3
5	213	I2C5_SCL	SCL5
	212	I2C5_SDA	SDA5
6	23	TP_I2C6_SCL	SCL6
	24	TP_I2C6_SDA	SDA6
7	205	CAM0_I2C7_SCL	SCL7

	206	CAM0_I2C7_SDA	SDA7
8	65	CAM1_I2C8_SCL	SCL8
	164	CAM1_I2C8_SDA	SDA8

设备默认已初始化所有 I2C 控制器，用户可以在*/sys/devices/platform/soc* 节点查看信息。如下图所示：

```
sh-5.2# ls -l | grep '\.i2c'
drwxr-xr-x  4 root root    0 May 30 22:42 11b20000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11c20000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11c21000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11d71000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11d72000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11d73000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11f30000.i2c
drwxr-xr-x  4 root root    0 May 30 22:42 11f31000.i2c
sh-5.2#
```

图 3：I2C 节点位置

已经配置完成的 I2C 控制器可以直接使用，具体配置信息如下所示。

7.2. 配置文件介绍

I2C 配置文件路径如下：

- *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts*
- *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8189.dtsi*

1. *mt8189.dtsi* 文件中的 I2C 控制器配置如下：

```
i2c0: i2c@11c20000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11c20000 0 0x1000>,
        <0 0x11300200 0 0x80>;
    interrupts = <GIC_SPI 274 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_e_clk CLK_IMPE_I2C0>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

```
i2c1: i2c@11c21000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11c21000 0 0x1000>,
        <0 0x11300300 0 0x80>;
    interrupts = <GIC_SPI 275 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_e_clk CLK_IMPE_I2C1>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

```
i2c2: i2c@11b20000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11b20000 0 0x1000>,
        <0 0x11300400 0 0x80>;
    interrupts = <GIC_SPI 276 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_ws_clk CLK_IMPWS_I2C2>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

```
i2c3: i2c@11d70000 {
    compatible = "mediatek,mt8188-i2c";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0 0x11d70000 0 0x1000>,
        <0 0x11300500 0 0x80>;
    interrupts = <GIC_SPI 277 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_s_clk CLK_IMPS_I2C3>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    status = "disabled";
};
```

```
i2c4: i2c@11d71000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11d71000 0 0x1000>,
        <0 0x11300600 0 0x80>;
    interrupts = <GIC_SPI 208 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_s_clk CLK_IMPS_I2C4>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

```
i2c5: i2c@11d72000 {
    compatible = "mediatek,mt8188-i2c";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0 0x11d72000 0 0x1000>,
        <0 0x11300700 0 0x80>;
    interrupts = <GIC_SPI 209 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_s_clk CLK_IMPS_I2C5>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    status = "disabled";
};
```

```
i2c6: i2c@11d73000 {
    compatible = "mediatek,mt8188-i2c";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0 0x11d73000 0 0x1000>,
        <0 0x11300800 0 0x80>;
    interrupts = <GIC_SPI 210 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_s_clk CLK_IMPS_I2C6>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    status = "disabled";
};
```

```
i2c7: i2c@11f30000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11f30000 0 0x1000>,
        <0 0x11300900 0 0x80>;
    interrupts = <GIC_SPI 211 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_en_clk CLK_IMPEN_I2C7>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

```
i2c8: i2c@11f31000 {
    compatible = "mediatek,mt8188-i2c";
    reg = <0 0x11f31000 0 0x1000>,
        <0 0x11300a00 0 0x80>;
    interrupts = <GIC_SPI 212 IRQ_TYPE_LEVEL_HIGH 0>;
    clocks = <&imp_iic_wrap_en_clk CLK_IMPEN_I2C8>,
        <&pericfg_ao_clk CLK_PERAO_DMA_B>;
    clock-names = "main", "dma";
    clock-div = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

备注

i2c3 默认未启用，用户若需启用 i2c3，将配置中的 *status="disabled"*改为 *status="ok"*即可。

```
&i2c3{
    pinctrl-names = "default";
    pinctrl-0 = <&i2c3_pins>;
    status = "disabled";
};
```

2. 以触摸屏挂载在 i2c6 为例，在 *mt8391-genio-720-evk.dts* 文件中，i2c6 控制器节点配置如下：

```
&i2c6{
    pinctrl-names = "default";
    pinctrl-0 = <&i2c6_pins>;
    status = "okay";
    clock-frequency = <400000>;
    /* touch device  */
    touchscreen@14 {
        compatible = "goodix,gt9271";
        reg = <0x14>;
        interrupt-parent = <&pio>;
        interrupts-extended = <&pio 11 IRQ_TYPE_EDGE_RISING>;
        irq-gpios = <&pio 11  GPIO_ACTIVE_HIGH>;
        reset-gpios = <&pio 10 GPIO_ACTIVE_HIGH>;
        AVDD28-supply = <&touch0_fixed_3v3>;
        VDDIO-supply = <&mt6359_vio18_ldo_reg>;
        pinctrl-names = "default";
        pinctrl-0 = <&touch_pins>;
        status = "disable";
    };

    cap_touch@38 {
        compatible = "mediatek,cap_touch" ;
        reg = <0x38> ;
        status = "okay" ;
    };
};
```

7.3. 内核中 I2C 客户端配置

7.3.1. 配置 I2C 客户端设备

以触摸屏挂载在 i2c6 为例：

进入 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 路径，配置内核中的 I2C 客户端设备。配置步骤如下：

1. 使能 i2c6 控制器；
2. 在 i2c6 节点下添加触摸屏设备节点；
3. 设置设备地址，例如：*reg = <0x38>*。其中 0x38 为触摸屏 I2C 设备地址。如下图所示：

```
&i2c6{
    pinctrl-names = "default";
    pinctrl-0 = <&i2c6_pins>;
    status = "okay";
    clock-frequency = <400000>;
    /* touch device  */
    touchscreen@14 {
        compatible = "goodix,gt9271";
        reg = <0x14>;
        interrupt-parent = <&pio>;
        interrupts-extended = <&pio 11 IRQ_TYPE_EDGE_RISING>;
        irq-gpios = <&pio 11  GPIO_ACTIVE_HIGH>;
        reset-gpios = <&pio 10 GPIO_ACTIVE_HIGH>;
        AVDD28-supply = <&touch0_fixed_3v3>;
        VDDIO-supply = <&mt6359_vio18_ldo_reg>;
        pinctrl-names = "default";
        pinctrl-0 = <&touch_pins>;
        status = "disable";
    };

    cap_touch@38 {
        compatible = "mediatek,cap_touch" ;
        reg = <0x38> ;
        status = "okay" ;
    };
};
```

7.3.2. 验证 I2C 客户端配置结果

配置完成后，执行 **cd /sys/bus/i2c/devices # ls** 命令查看配置结果。按照上述配置，如查询结果显示 *6-0038* 则表示 I2C 客户端设备已配置成功，即触摸屏已成功挂载在 i2c6 下，如下图所示：

```
sh-5.2# ls
1-0048  4-004e  5-0022  8-003c  i2c-1  i2c-4  i2c-6  i2c-8
1-0068  4-006b  6-0038  i2c-0   i2c-2  i2c-5  i2c-7  i2c-9
sh-5.2#
```

7.4. 内核 I2C 接口函数

内核中常用的标准 I2C 接口函数如下，具体使用案例请参考源码。

```
//从设备读取一个字节数据。
s32 i2c_smbus_read_byte(const struct i2c_client *client);
//向设备写入一个字节数据。
s32 i2c_smbus_write_byte(const struct i2c_client *client, u8 value);
//从设备指定偏移处读取一个字节数据。
s32 i2c_smbus_read_byte_data(const struct i2c_client *client, u8 command);
//向设备指定偏移处写入一个字节数据。
s32 i2c_smbus_write_byte_data(const struct i2c_client *client, u8 command, u8 value);
//从设备指定偏移处读取两个字节数据。
s32 i2c_smbus_read_word_data(const struct i2c_client *client, u8 command);
//向设备指定偏移处写入两个字节数据。
s32 i2c_smbus_write_word_data(const struct i2c_client *client, u8 command, u16 value);
//I2C 发送一次数据
int i2c_master_send(const struct i2c_client *client, const char *buf, int count);
//I2C 接收一次数据
int i2c_master_recv(const struct i2c_client *client, char *buf, int count);
//I2C 收发数据
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num);
```

7.5. 用户空间中操作 I2C 示例

7.5.1. 调用 I2C 相关接口读写数据

以下代码展示如何调用 I2C 相关接口读取和写入数据：

```
#define CHIP_ADDR 0x50
int main(void) {
int fd = open("/dev/I2C-3", O_RDWR);
if (fd < 0) {
        printf("open /dev/I2C-3 failed\n");
        return -1;
        }
        if (ioctl(fd, I2C_SLAVE_FORCE, CHIP_ADDR) < 0) {
            printf("ioctl: set slave address failed\n");
            return -1;
            }
        struct I2C_msg msg; unsigned char rddata;
        unsigned char rdaddr[1] = {0x10};
```

```
        unsigned char wrbuf[2] = {0x10, 0x3c};

        write(fd, wrbuf, 2);
        sleep(1); write(fd, rdaddr, 1);
        read(fd, &rddata, 1));
        printf("rddata: %02x\n", rddata);

        close(fd);
        return0;
```

7.5.2. 调用底层 ioctl 接口读写数据

以下代码展示如何调用底层 ioctl 接口读取和写入数据：

```
#define CHIP_ADDR 0x50
int main(void)
{
    int fd =open("/dev/I2C-3", O_RDWR);
    if (fd < 0) {
        printf("open /dev/I2C-3 failed\n");
        return -1;
        }
    struct I2C_msg msg; unsigned char rddata;
    unsigned char rdaddr[1] = {0x10};
    unsigned char wrbuf[2] = {0x10, 0x3c};
    struct I2C_rdwr_ioctl_data ioctl_data;
    struct I2C_msg msgs[2];
    msgs[0].addr= CHIP_ADDR;
    msgs[0].len = 2; msgs[0].buf = wrbuf;
    ioctl_data.nmsgs = 1;
    ioctl_data.msgs = &msgs[0];
    ioctl(fd, I2C_RDWR, &ioctl_data);
    usleep(100);
    msgs[0].addr= CHIP_ADDR; msgs[0].len = 1;
    msgs[0].buf = rdaddr;
    msgs[1].addr = CHIP_ADDR;
    msgs[1].flags |= I2C_M_RD;
    msgs[1].len = 1;
    msgs[1].buf = &rddata;
    ioctl_data.nmsgs = 2;
    ioctl_data.msgs = msgs;
    ioctl(fd, I2C_RDWR, &ioctl_data));
    printf("rddata: %02x/n", rddata);
    close(fd);
```

```
    return0;
}
```

8 GPIO 功能

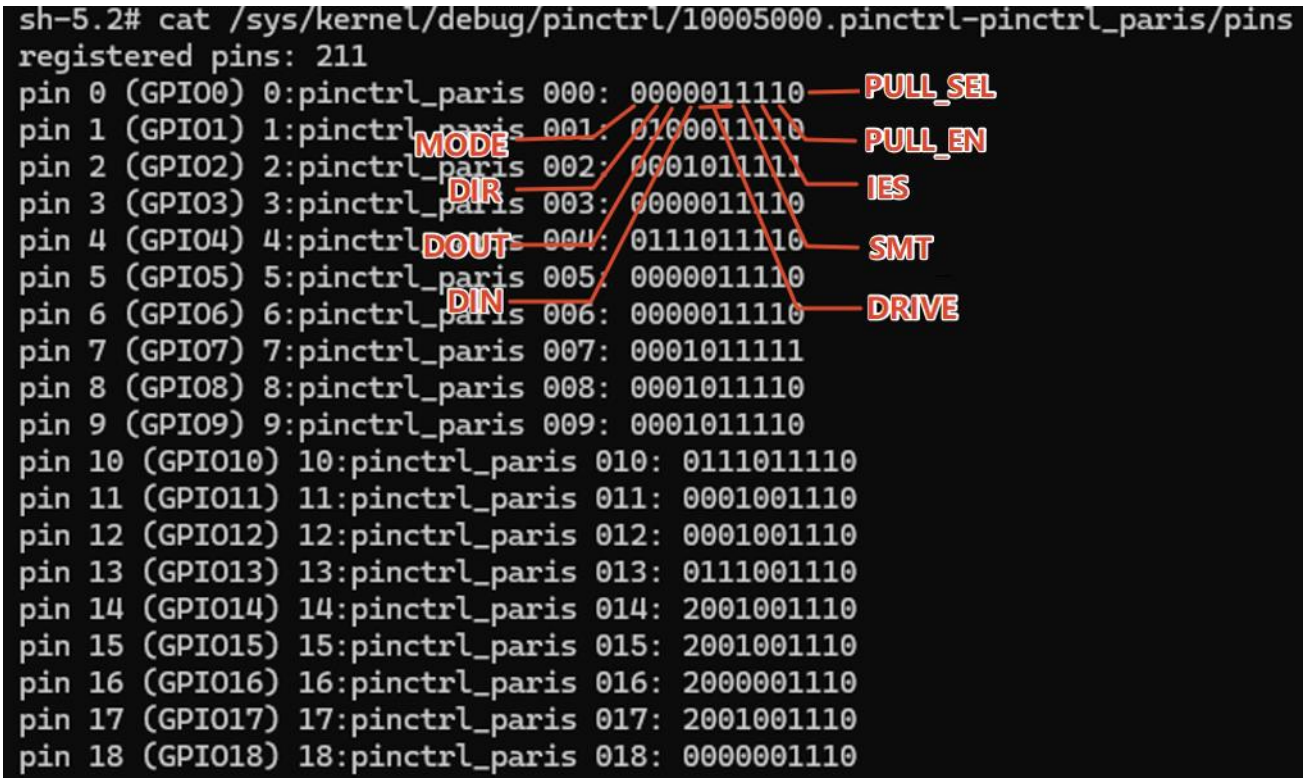
本章节主要介绍如何配置和操作设备的 GPIO 功能。

8.1. GPIO 简介

设备共有 183 个 GPIO 端口。可以在设备中通过如下命令查询到当前 GPIO 的状态。

```
cat /sys/kernel/debug/pinctrl/10005000.pinctrl-pinctrl_paris/pins
```

返回结果如下所示。



每一行数据表示一个 GPIO 配置信息，重点是最后的数字段，从左到右分别表示的含义如下表所示：

数字位	名称	含义
1	MODE	当前 PIN 处于的模式
2	DIR	0 输入引脚 1 输出引脚
3	DOUT	输出值。 1 高电平 0 低电平
4	DIN	输入值。 1 高电平 0 低电平
[5,6]	DRIVE	驱动能力，一般可取值 0~7，可占用两位数字
7	SMT	使能施密特触发器。 1 使能 0 禁能
8	IES	输入使能。 1 输入信号有效 0 输入信号无效
9	PULL_EN	只对输入引脚有效，上下拉使能。 1 使能 0 禁能
10	PULL_SEL	只对输入引脚有效，上下拉配置。 1 上拉 0 下拉
[11,12]	(R1) (R0)	1 0 表示使能 R1，禁能 R0 0 1 表示禁能 R1，使能 R0 1 1 表示使能 R1，使能 R0 省略表示禁能 R1 和 R0

8.2. 在内核中操作 GPIO

本章节介绍如何在内核设备树文件以及驱动代码中配置及操作 GPIO。

8.2.1. 在设备树中定义 GPIO 编号

在设备树文件 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 中定义 GPIO 编号。

```
gpiolib_test:gpio_test{
        compatible="gpiolib_usage_test";
        nr-gpios = <&pio 11 GPIO_ACTIVE_HIGH>;
        status = "okay";
}
```

8.2.2. 在内核驱动中包含 GPIO 头文件

在内核驱动中包含如下所示的 GPIO 头文件：

```
#include <linux/gpio.h>
#include <linux/of_gpio.h>
```

8.2.3. 从设备树定义中获取 GPIO 编号

调用以下函数获取设备树中定义的 GPIO 编号：

```
struct gpio_desc *test_gpio;
test_gpio = devm_gpiod_get(dev,"nr",GPIOD_OUT_HIGH);
```

8.2.4. 配置 GPIO 为输入模式

```
gpiod_direction_input(test_gpio);
```

8.2.5. 配置 GPIO 为输出模式

```
gpiod_direction_output(test_gpio,0);
```

8.2.6. 在内核驱动中设置 GPIO 输出电平

若当前 GPIO 被配置为输出模式，则可以修改 GPIO 电平，如下所示：

```
//修改当前 GPIO 输出低电平。
gpiod_set_value(gpio_number, 0);

//修改当前 GPIO 输出高电平。
gpiod_set_value(gpio_number, 1);
```

8.3. 使用 pinctrl 配置 GPIO

本章节介绍如何在内核设备树文件以及驱动代码中使用 pinctrl 配置及操作 GPIO。

8.3.1. 在设备树中定义 GPIO 的 pinctrl 属性

在设备树文件 *sources/linux-mtk/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts* 中定义 GPIO 的 pinctrl 属性。示例如下：

```
&cap_touch {
	//.......
	pinctrl-2 = <&ctp_pins_eint_output0>;
	//.....
};
//....
&pio {
	ctp_pins_eint_output0: eintoutput0 {
		pins_cmd_dat {
			pinmux = <PINMUX_GPIO0__FUNC_GPIO0>;
			output-low;
		};
	};
};
```

其中 *pins_cmd_dat* 是关于此 GPIO 的配置信息，如上述信息所示，配置 GPIO0 为 GPIO 模式，输出低电平。

8.3.2. 在内核驱动中包含头文件

在内核驱动中包含头文件 *consumer.h*。

```
#include <linux/pinctrl/consumer.h>
```

8.3.3. 在内核驱动中获取 pinctrl 及其状态

```
static struct pinctrl *pctrl;
static struct pinctrl_state *eintoutput0;
pctrl= devm_pinctrl_get(&pdev->dev);
if (IS_ERR_OR_NULL(pctrl)) {
	pr_err("linuxc get pinctrl failed\n");
	return -EINVAL;
```

```
}

gpio_ctrl1 = pinctrl_lookup_state(pctrl, “eintoutput0”);
if (IS_ERR_OR_NULL(gpio_ctrl1)) {
        pr_err(“linuxc lookup pinctrl eintoutput0 state failed\n”);
        return -EINVAL;
}
```

8.3.4. 在内核驱动中选择 pinctrl 状态

在内核驱动中选择 pinctrl 状态，示例如下：

```
int ret;

ret = pinctrl_select_state(pctrl, eintoutput0);
if (ret) {
        pr_err(“linuxc select pinctrl eintoutput0 state failed\n”);
        return -EBUSY;
}
```

9 PCIe 功能

9.1. 功能概述

QSM603FCP 设备支持一路 PCIe 接口，链路形态为 1-lane，目标速率为 Gen2（5.0 GT/s）。该设备通过 PCIe 接口连接 YT6801 网卡芯片，由 YT6801 提供三个 RJ45 网口，包括一个千兆网口和两个百兆网口。

PCIe 连接关系如下：

QSM603FCP（Root Complex） → PCIe → YT6801

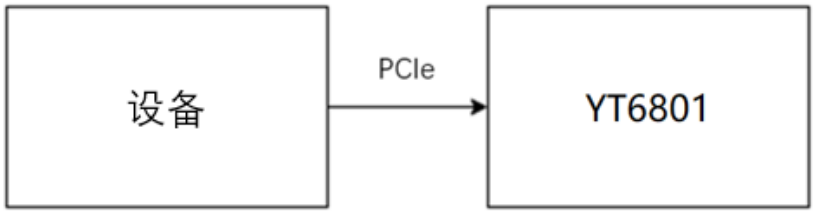


图 4：设备 PCIe 至 YT6801 通路图

9.2. PCIe 能力说明

设备支持以下 PCIe 特性：

1. 符合 PCI Express Base Specification Revision 3.0。
2. 支持链路速率：2.5 GT/s（Gen1）、5.0 GT/s（Gen2）。
3. 支持 ASPM（L0s/L1）与 L1 PM Substates（L1PMSS）。
4. 支持 MSI 中断和 PCI 传统 INTx 中断。

备注

实际协商的链路速率由 RC（Root Complex）和 EP（Endpoint）双方共同决定。若 RC 或 EP 一方仅支持 Gen1，则链路速率将协商为 2.5 GT/s。

9.3. PCIe 控制器节点

SoC 公共 DTSI 文件中已定义 PCIe 控制器节点，定义路径如下：

- *target/mtk/mt8371/source/linux-mtk-6.8/arch/arm64/boot/dts/mediatek/mt8189.dtsi*

PCIe 控制器节点定义如下：

```
pcie: pcie@112f0000 {
        device_type = "pci";
        compatible = "mediatek,mt8189-pcie",
                     "mediatek,mt8192-pcie";
        reg = <0 0x112f0000 0 0x4000>;
        reg-names = "pcie-mac";
        linux,pci-domain = <0>;
        #address-cells = <3>;
        #size-cells = <2>;
        interrupts = <GIC_SPI 467 IRQ_TYPE_LEVEL_HIGH 0>;
        bus-range = <0x00 0xff>;
        ranges = <0x82000000 0 0x30000000
                  0x0 0x30000000 0 0x04000000>;

        status = "disabled";

        clocks = <&pericfg_ao_clk CLK_PERAO_AXI>,
                 <&pericfg_ao_clk CLK_PERAO_AHB_APB>,
                 <&pericfg_ao_clk CLK_PERAO_TL>,
                 <&pericfg_ao_clk CLK_PERAO_REF>,
                 <&topckgen_clk CLK_TOP_F26M_CK_EN>;

        phys = <&pcieport PHY_TYPE_PCIE>;
        phy-names = "pcie-phy";
        power-domains = <&scpsys MT8189_POWER_DOMAIN_PCIE>;

        #interrupt-cells = <1>;
        interrupt-map-mask = <0 0 0 7>;
        interrupt-map = <0 0 0 1 &pcie_intc 0>,
                        <0 0 0 2 &pcie_intc 1>,
                        <0 0 0 3 &pcie_intc 2>,
                        <0 0 0 4 &pcie_intc 3>;
        pcie_intc: interrupt-controller {
                interrupt-controller;
                #address-cells = <0>;
```

```
            #interrupt-cells = <1>;
        };
};
```

在如下路径中添加供电使能，完成 YT6801 芯片供电：

- *target/mtk/mt8371/source/linux-mtk-6.8/arch/arm64/boot/dts/mediatek/mt8391-genio-720-evk.dts*

```
&pcie {
    pinctrl-names = "default";
    pinctrl-0 = <&pcie_default_pins>;
    vddpe-3v3-supply = <&vdc_3v3reg>;
    vdd-3v3-supply = <&asm1182e_3v3reg>;
    vdd1v2-supply = <&asm1182e_1v2reg>;
    lan1-3v3-gpios = <&nca9555 13 GPIO_ACTIVE_HIGH>;
    lan2-3v3-gpios = <&nca9555 14 GPIO_ACTIVE_HIGH>;
    status = "okay";
};
```

9.4. PCIe 控制器驱动

设备的 PCIe 控制器驱动位于：*sources/linux-mtk/drivers/pci/controller/pcie-mediatek-gen3.c*。

驱动实现包含以下 PCIe 相关功能：

- INTx 中断域初始化与处理中断流程。
- MSI/MSI-X 中断域初始化、MSI message 地址配置与分发。
- 提供 LTSSM 状态信息，包括 L0s、L1.entry 和 L1.idle 等状态，可用于链路省电状态的调试。

10 附录 参考文档和术语缩写

表 6：参考文档

文档名称
[1] Quectel_QSM603FCP_Ubuntu_编译&烧录指导

表 7：术语缩写

缩写	英文全称	中文全称
ADB	Android Debug Bridge	安卓调试桥
ADC	Analog-to-Digital Converter	模数转换器
API	Application Programming Interface	应用程序编程接口
ASPM	Active State Power Management	主动状态电源管理
CAN	Controller Area Network	控制器局域网
DC	Direct Current	直流电
DMA	Direct Memory Access	直接存储器访问
DRD	Dual Role Device	双角色设备
DTS	Device Tree Source	设备树源文件
DTSI	Device Tree Source Include	包含设备树源文件
EP	Endpoint	端点设备
GPIO	General-Purpose Input/Output	通用型输入/输出
I/O	Input/Output	输入、输出
I2C	Inter-Integrated Circuit	集成电路总线
ID	Identifier	标识符
IIO	Industrial I/O Subsystem	工业 I/O 子系统
L1PMSS	L1 Power Management Substates	L1 电源管理子状态

LTSSM	Link Training and Status State Machine	链路训练与状态状态机
MSI	Message Signaled Interrupts	消息信号中断
MTP	Media Transfer Protocol	多媒体传输协议
PBL	Primary Boot Loader	主引导加载程序
PCI	Peripheral Component Interconnect	外围部件互连
PCIe	Peripheral Component Interconnect Express	快捷外围部件互连标准
PHY	Physical	端口物理层
PTP	Picture Transfer Protocol	图片传输协议
RC	Root Complex	根复合体
RNDIS	Remote Network Driver Interface Specification	远程网络驱动接口规范
SPI	Serial Peripheral Interface	串行外设接口
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发传输器
USB	Universal Serial Bus	通用串行总线